

Reasoning by Exploration: A Unified Approach to Retrieval and Generation over Graphs

Haoyu Han
Michigan State University
East Lansing, MI, USA
hanhaoy1@msu.edu

Yu Wang
University of Oregon
Eugene, OR, USA
yuwang@uoregon.edu

Li Ma^{*}
Michigan State University
East Lansing, MI, USA
mali13@msu.edu

Kai Guo^{*}
Michigan State University
East Lansing, MI, USA
guokai1@msu.edu

Yucheng Chu
Michigan State University
East Lansing, MI, USA
chuyuch2@msu.edu

Jiliang Tang
Michigan State University
East Lansing, MI, USA
tangjili@msu.edu

Harry Shomer
University of Texas at Arlington
Arlington, TX, USA
harry.shomer@uta.edu

Hang Li
Michigan State University
East Lansing, MI, USA
lihang4@msu.edu

Abstract

Reasoning over structured graphs remains a fundamental challenge for Large Language Models (LLMs), particularly when scaling to large graphs. Existing approaches typically follow the retrieval-augmented generation (RAG) paradigm: first retrieving subgraphs relevant to the query and then generating answers conditioned on the retrieved subgraphs. However, such two-phase pipelines often struggle to faithfully incorporate graph structure, since the generation process is ultimately constrained by the quality and completeness of the retrieved subgraph. Although many advanced retrievers have been proposed recently to mitigate this issue, they are usually tailored to the training graphs and generalize poorly to unseen graphs, which limits their practical applicability. In this work, we propose Reasoning by Exploration (RoE), a novel approach that unifies retrieval and generation by framing reasoning over graphs as a process of graph exploration. At each step, the LLM selects candidate nodes and edges to explore, gradually constructing reasoning paths and generating answers along the way. To enable effective exploration, RoE is trained in two stages: supervised fine-tuning (SFT) on gold reasoning paths, followed by reinforcement learning (RL) to enhance exploration effectiveness and generalization. Experiments on benchmark datasets demonstrate that RoE achieves substantial overall improvements over baselines, while also generalizing effectively to unseen graphs. The code can be found at <https://github.com/haoyuhan1/RoE>.

CCS Concepts

• Information systems → Language models; Question answering; • Computing methodologies → Reinforcement learning.

^{*}Corresponding Authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792684>

Keywords

Large Language Models, Graph Reasoning, Retrieval-Augmented Generation, Multi-hop Question Answering

ACM Reference Format:

Haoyu Han, Kai Guo, Harry Shomer, Yu Wang, Yucheng Chu, Hang Li, Li Ma, and Jiliang Tang. 2026. Reasoning by Exploration: A Unified Approach to Retrieval and Generation over Graphs. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792684>

1 Introduction

Large language models (LLMs) [5, 26, 51] have demonstrated impressive abilities across a wide range of natural language processing tasks. To further enhance their reliability and factuality, Retrieval-Augmented Generation (RAG) [14, 25, 30, 56] has become a widely adopted paradigm. By retrieving external knowledge before generating responses, RAG has achieved strong performance in diverse domains, such as healthcare [63], law [59], finance [69], and education [7]. However, most existing RAG systems operate over unstructured text, retrieving passages through lexical or semantic similarity, and therefore do not fully leverage the relational structure inherent in many forms of knowledge.

Graphs provide a natural representation for structured knowledge, capturing rich relational dependencies among entities. They are widely used in domains such as knowledge bases, social networks, and scientific applications including biology and chemistry [36, 55, 61]. Recently, Graph Retrieval-Augmented Generation (GraphRAG) has emerged as a promising way to integrate such graph-structured knowledge into LLMs [18, 28, 41]. There are mainly two steps in GraphRAG: first, a retriever selects query-relevant content from the entire graph, where the retrieved units may be nodes, paths, or subgraphs; and then, a generator, usually an LLM, produces answers conditioned on the retrieved subgraphs. This framework has been successfully applied to tasks such as knowledge graph question answering (KGQA) [65, 66, 72], where often requires multi-step reasoning over the graphs.

While efficient, this two-phase pipeline may constrain reasoning quality due to the disjoint optimization of retriever and generator. **First**, the retriever often ignores the specific needs of the generator, which can lead to missing critical evidence or introducing irrelevant noise. For example, consider the multi-hop question “Who are the nephews of Winston Churchill?”. This query may have multiple answers and can be reached through multiple reasoning paths, such as “Churchill $\rightarrow$ sibling $\rightarrow$ child” or “Churchill $\rightarrow$ parent $\rightarrow$ child $\rightarrow$ child”. Heuristic-based retrievers [13, 65] (e.g., retrieving fixed k -hop subgraphs) or GNN-based retrievers [32, 37] may include a large amount of irrelevant information about Churchill, overwhelming the generator with noise [16]. In contrast, LLM-based retrievers [35, 60] may generate only a limited set of reasoning paths, leading to missing valid entities and incomplete answers, as shown in Section 3. **Second**, retrievers or generators trained on specific graphs or datasets usually generalize poorly to unseen graphs and question types as shown in Section 3. For instance, a path-based retriever tuned on a movie knowledge base may learn to generate reasoning paths primarily around relations such as “actor $\rightarrow$ film $\rightarrow$ director”, but it may struggle to generalize when applied to graphs in other domains with very different relational structures, such as biomedical graphs involving “gene $\rightarrow$ protein $\rightarrow$ disease” paths. This dependence leads to sharp performance drops when deployed on unseen graphs, limiting the robustness and real-world applicability of current GraphRAGs.

To address these limitations, we propose Reasoning by Exploration (RoE), a unified framework that leverages the reasoning ability of LLMs to perform retrieval and generation simultaneously within the same model. Instead of separating the two stages, RoE frames reasoning as a step-by-step graph exploration process, where the model incrementally expands nodes and edges while constructing reasoning paths and generating answers in a unified manner. In this way, RoE has the access to the whole graph. This design directly mitigates the problem of missing or noisy retrieval, since the model actively decides which parts of the whole graph to explore based on its evolving reasoning process, instead of relying on a static retriever. However, it faces two main challenges. *First*, how can we equip LLMs, which pretrained primarily on unstructured text, with the ability to perform structured graph exploration? *Second*, how can we ensure that the model learns generalizable exploration strategies rather than memorizing specific graph patterns, so that it can adapt to unseen graphs? To tackle these challenges, we adopt a two-stage training strategy. In the first stage, we construct gold exploration trajectories and apply supervised fine-tuning (SFT) to teach the LLM to expand nodes and edges step by step. In the second stage, we employ reinforcement learning (RL) to further refine the exploration policy, encouraging the model to efficiently discover valid reasoning paths while improving its ability to generalize across diverse and unseen graphs. Together, this training paradigm enables RoE to combine explicit supervision with adaptive learning signals, leading to more faithful reasoning and stronger generalization. We conduct extensive experiments on multiple multi-hop reasoning benchmarks, showing that RoE consistently outperforms strong state-of-the-art baselines and exhibits robust generalization to unseen graphs and question types across different backbone sizes.

2 Related Works

2.1 GraphRAG

GraphRAG [17, 18, 29, 41] aims to retrieve relevant information from external graphs to improve the performance and reduce the hallucination of LLMs. Compared to traditional RAG systems [14, 30] that operate on unstructured text, GraphRAG leverages the relational structure of graphs, where nodes represent entities and edges encode relations between nodes. Similar to RAG, GraphRAG generally consists of two main components: a retriever, which selects query-relevant graph content, and a generator, typically an LLM, which produces answers based on the retrieved subgraphs. To accommodate the graph structure, there are mainly three types of retrievers:

Heuristic-based Retriever. Heuristic-based retrievers typically begin by using similarity-based methods [43, 58] or entity and relation extraction techniques [2, 13] to identify seed nodes and edges that are relevant to the query. These seeds are then expanded by retrieving their k -hop neighbors [6, 66], forming a candidate subgraph for reasoning. A representative example is G-Retriever [21], which first leverages embedding models to retrieve the most similar nodes and edges to the query, and then applies the Prize-Collecting Steiner Tree (PCST) algorithm to extract a compact subgraph.

GNN-based Retriever. GNN-based retrievers [12, 32] typically train Graph Neural Networks (GNNs) [36, 61] to model retrieval as a node classification task, where the correct answer entities are labeled as positive and others as negative. For example, GNN-RAG [37] first trains a GNN model and then retrieves the nodes with the highest probability scores as candidate answers, while also identifying the shortest paths connecting the question entities with these candidates to construct supporting evidence.

LLM-based Retriever. LLM-based retrievers leverage the reasoning ability of LLMs to generate retrieval plans instead of relying solely on graph embeddings or heuristics. For example, they may prompt an LLM to produce SQL queries over structured knowledge bases [23] or to explicitly plan multi-hop paths from the query entities to answers [76]. A representative method is RoG [35], which fine-tunes an LLM to generate reasoning relation paths and then retrieves the corresponding paths from the graph.

While these retrievers have demonstrated success across different tasks, they remain unaware of the generation process, which often leads to incomplete retrieval and the introduction of substantial noise as shown in Section 3. Moreover, training-based methods are usually tailored to a specific graph, further limiting their ability to generalize to unseen graphs. Instead, the proposed RoE unifies retrieval and generation within the same process, enabling the model to actively explore the graph while constructing reasoning paths and generating answers in an integrated manner.

2.2 Reasoning of Large Language Models

Large Language Models (LLMs) have demonstrated strong performance on natural language tasks [5, 19, 26, 51], but they may struggle with more complex reasoning problems [39, 74]. To further improve their reasoning ability, early works leverage prompting-based methods, such as Chain-of-Thought [57], Tree-of-Thought [64] and Graph-of-Thought [3], which encourage models to generate and

organize intermediate reasoning steps during inference. Think-on-graph (ToG) [48] also follow this paradigm to prompt LLM reason on the graph. While effective, these methods are often sensitive to prompt design and computationally inefficient. Building on this, more recent approaches [45, 62] focus on supervised fine-tuning (SFT) with curated reasoning traces [10, 33] and reinforcement learning (RL) with process or outcome rewards [42, 70]. These methods allow models to internalize reasoning strategies rather than relying only on inference-time prompting. However, most of these advances focus on text-only tasks, whereas reasoning over graph-structured knowledge introduces additional challenges, requiring models to explore nodes, edges, and multi-hop paths. Although several works [34, 53, 68] have attempted to adapt LLMs for graph reasoning tasks, most of them explicitly decompose complex queries into simpler sub-queries and rely on external retrievers. It remains underexplored how to enable LLMs to automatically explore graphs and generate answers in a unified manner.

In this paper, we follow previous studies [21, 35, 48] and leverage the Knowledge Graph Question Answering (KGQA) task to evaluate the effectiveness of graph reasoning, as it inherently involves multi-hop relational reasoning and explicit interaction with graph structures to obtain correct answers.

3 Preliminary

In this section, we first provide definitions, and then analyze the retrievers adopted in current popular GraphRAG methods.

3.1 Definitions

Knowledge Graphs (KGs). Let $\mathcal{G} = (\mathcal{V}, \mathcal{R}, \mathcal{E})$ be a knowledge graph, where $\mathcal{V}$ denotes entities, $\mathcal{R}$ relation types, and $\mathcal{E}$ directed edges. Each edge is represented as a triple (h, r, t) with $h, t \in \mathcal{V}$ and $r \in \mathcal{R}$. For any node $v \in \mathcal{V}$, we denote its neighboring entities, incident relations, and connected edges as $\mathcal{N}_v$, $\mathcal{R}_v$, and $\mathcal{E}_v$, respectively. In this paper, we also augment the knowledge graph with inverse edges. Specifically, for each triple $(h, r, t) \in \mathcal{E}$, we add its inverse edge $(t, r_{\text{inverse}}, h)$, following prior works [11, 27].

Knowledge Graph Question Answering (KGQA). KGQA is the task of answering natural language questions by reasoning over a knowledge graph. Given a question and a KG, $\mathcal{G} = \{\mathcal{V}, \mathcal{R}, \mathcal{E}\}$, the goal is to identify the correct entity or set of entities in $\mathcal{V}$ that answer the query. This typically requires multi-hop reasoning, where the model must traverse nodes and relations along valid paths (e.g., $(h, r_1, v_1), (v_1, r_2, t)$) to reach the answer.

Due to the large size of knowledge graphs, which cannot fit into the limited context window of LLMs at once, most KGQA methods (i.e., GraphRAG) first retrieve a subgraph relevant to the query and then restrict the LLM’s reasoning to this subgraph. However, in this two-stage pipeline, the retriever and generator are usually optimized independently and executed sequentially. In the following subsections, we present experiments that illustrate the limitations of this design, focusing on the quality of the retrieved subgraphs, which can greatly affect the overall generation performance.

3.2 Experimental Settings

To conduct a comprehensive evaluation, we select one representative method from each retriever category as introduced in Section 2:

G-Retriever [21] for heuristic-based methods, GNN-RAG [37] for GNN-based methods, and RoG [35] for LLM-based methods.

We select two widely used datasets: WebQSP [67], which primarily contains questions requiring one- or two-hop reasoning, and CWQ [49], which consists of more complex questions that require multi-hop reasoning. For each question, there can be multiple correct answers. On average, the number of answers per test question is 10.20 for WebQSP and 1.89 for CWQ. For the heuristic-based method, i.e., G-Retriever, no training of the retriever is required. In contrast, both GNN-RAG and RoG require training. Specifically, for GNN-RAG, we train a GNN model as the retriever on each dataset, while for RoG we fine-tune the Llama-3.1-8B-Instruct model [52] to serve as the retriever on each dataset. To evaluate retriever performance, we adopt three metrics: **Hit**, which measures whether at least one correct answer is included in the retrieved content; **Recall**, which measures the proportion of ground-truth answers retrieved for each question; and **Precision**, which measures the ratio of correctly retrieved entities among all retrieved entities.

3.3 In-Distribution Retriever Performance

In this subsection, we aim to measure the performance of different retrievers when they are both trained and tested on the same dataset (i.e., WebQSP or CWQ). The performance of the three selected methods is shown in Figure 1a, where we also include the ground-truth Hit and Recall as upper bounds, since the underlying graph does not contain all the answers for every question. From the results, we observe that none of the retrievers achieve perfect performance. For example, the Recall and Precision of G-Retriever on the CWQ dataset are only 38.65 and 5.14, respectively, indicating that many correct answers are missing while substantial noise is included in the retrieved subgraphs. Among the three methods, GNN-RAG performs best, but it still shows around a 10% gap compared to the ground-truth Hit upper bound on the CWQ dataset. This demonstrates that the *limitations of current retrievers will substantially constrain downstream generation performance*.

3.4 Cross-Distribution Generalization

In Section 3.3, we mainly studied the retriever performance when training and testing on the same dataset. However, in real-world applications it is often impractical to train a separate retriever for every new graph. In this subsection, we therefore explore the generalization ability of pretrained retrievers. Instead of fine-tuning the retrievers for each dataset as in GNN-RAG [37] and RoG [35], we directly apply a retriever pretrained on one dataset to test on the other, in order to evaluate its cross-distribution generalization.

We first conduct experiments on the WebQSP and CWQ datasets, which are both built on Freebase KG [4] but differ substantially in question distribution. WebQSP consists of short, naturally collected questions that require 1-2-hop reasoning, whereas CWQ contains synthetically expanded questions with up to 4 hops and more complex linguistic structures such as conjunctions and comparatives. As shown in Figure 1b, both RoG and GNN-RAG suffer a significant performance drop under the cross-distribution setting. For instance, the Hit of GNN-RAG decreases from 92.07 to 55.09, and its Recall drops from 85.43 to 31.68 on WebQSP when using the retriever trained on CWQ, performing even worse than the heuristic-based G-Retriever, which requires no retriever pretraining.

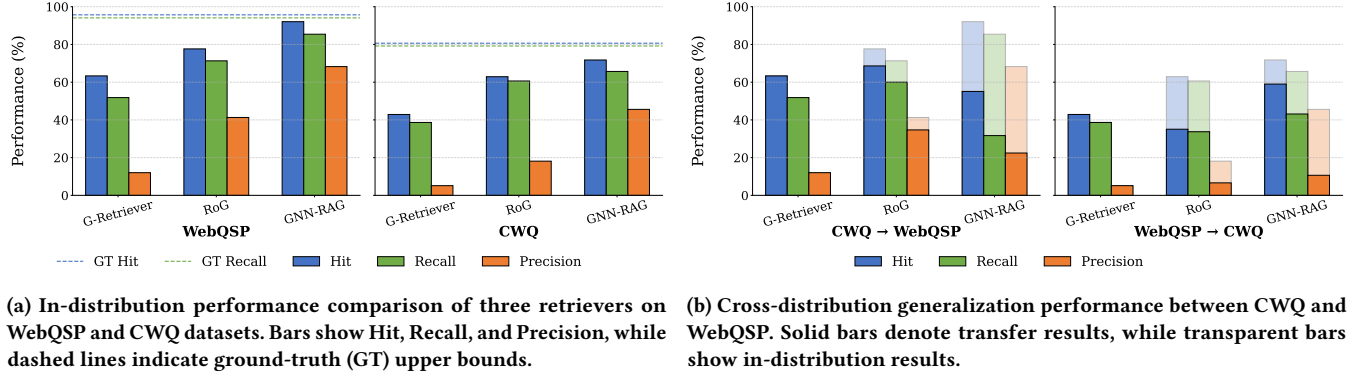


Figure 1: Retrieval and generalization performance of different methods on WebQSP and CWQ datasets.

We further evaluate cross-graph generalization by applying retrievers trained on WebQSP to datasets with different knowledge graphs, including MetaQA [73], which uses the WikiMovies KG, and PathQuestion [75], which uses a Freebase subset with altered relation names. As shown in Table 1, RoG fails to retrieve any valid content, while GNN-RAG retrieves overly large subgraphs with very low precision.

Table 1: Cross-graph generalization performance.

	Webqsp->MetaQA			Webqsp->PathQuestion		
	Hit	Recall	Precision	Hit	Recall	Precision
RoG	0	0	0	0	0	0
GNN-RAG	38.3	13.36	0.02	63.8	51.7	0.05

Overall, these results show that representative GraphRAG retrievers generalize poorly. We attribute this to two major factors:

- (1) **Distribution gap in questions.** Retrievers are typically trained to recognize the lexical and structural patterns of questions within a single dataset (e.g., WebQSP or CWQ). When the question style or reasoning composition changes, such as shifting from short, 1-2-hop natural questions to complex, synthetic multi-hop questions, the retriever fails to capture the correct reasoning paths and relevant entities. This suggests that retrievers overfit to dataset-specific surface forms rather than learning transferable reasoning patterns.
- (2) **Distribution gap in graphs.** Differences in the underlying knowledge graphs, such as relation vocabulary, and entity type distributions, further limit generalization. GNN-based retrievers depend on node and edge embeddings learned from a specific graph topology, while LLM-based retrievers (e.g., RoG) rely on relation paths that may not align across graphs. As a result, both classes of retrievers struggle to adapt when transferred to new graphs with different relation semantics or structural patterns.

3.5 Discussions

From Section 3.3, we observe that a substantial portion of answers cannot be retrieved by existing retrievers. Consequently, even strong generators fail to produce these correct answers since the necessary evidence is missing from the retrieved subgraphs. This highlights a key limitation of current two-stage GraphRAG paradigms—the retriever and generator operate independently and cannot interact with each other on the graph during reasoning.

The results in Section 3.4 further suggest that existing retrievers are over-specialized to their training datasets and knowledge graphs. They lack the capability to adjust their retrieval behavior based on the evolving environment or intermediate reasoning outcomes. *This static retrieval behavior prevents effective adaptation to unseen question types and graph structures.*

These insights motivates a new perspective on GraphRAG: rather than decoupling retrieval and generation or retrieving all potentially relevant content at once, the model should be able to **interact with the whole graph step by step**, selecting and exploring new nodes and relations as reasoning progresses for the question. Such a reasoning-driven retrieval process enables the model to dynamically incorporate feedback from intermediate steps, allowing it to generalize across diverse questions and graph structures by learning to *reason and explore* instead of memorizing fixed retrieval patterns.

4 Method

Motivated by the limitations of existing GraphRAG methods identified in Section 3, we propose Reasoning by Exploration (RoE), a unified framework that integrates retrieval and generation within a single process, as illustrated in Figure 2. Unlike traditional two-stage GraphRAG pipelines, where a retriever extracts a static subgraph before reasoning begins, RoE enables a single LLM to actively explore the entire knowledge graph step by step.

4.1 Exploration Formulation

We formulate reasoning in RoE as an **exploration process** over a knowledge graph, where the LLM incrementally traverses the graph to construct reasoning paths that lead to correct answers. Formally, we formulate the exploration process as follows:

Given a question q and a knowledge graph $\mathcal{G} = \{\mathcal{V}, \mathcal{R}, \mathcal{E}\}$, the model begins from one or more **seed entities** $\mathcal{V}_0$ that are linked to entities mentioned in the question. We expect the model to explore one-hop neighbors from the currently explored nodes at each step, progressively expanding the reasoning space. Specifically, at each step d , the model maintains a set of **current reasoning paths**:

$$\mathcal{P}_d = \{p_1^d, p_2^d, \dots, p_n^d\},$$

where each path p_i^d is defined as $p_i^d = [v_0, r_1, v_1, \dots, r_k, v_k]$, starting from a seed entity $v_0 \in \mathcal{V}_0$ and expanding through a sequence of relations and entities. We define **frontier nodes** as

$$\mathcal{F}_d = \{v_k \mid p_i^d \in \mathcal{P}_d\},$$

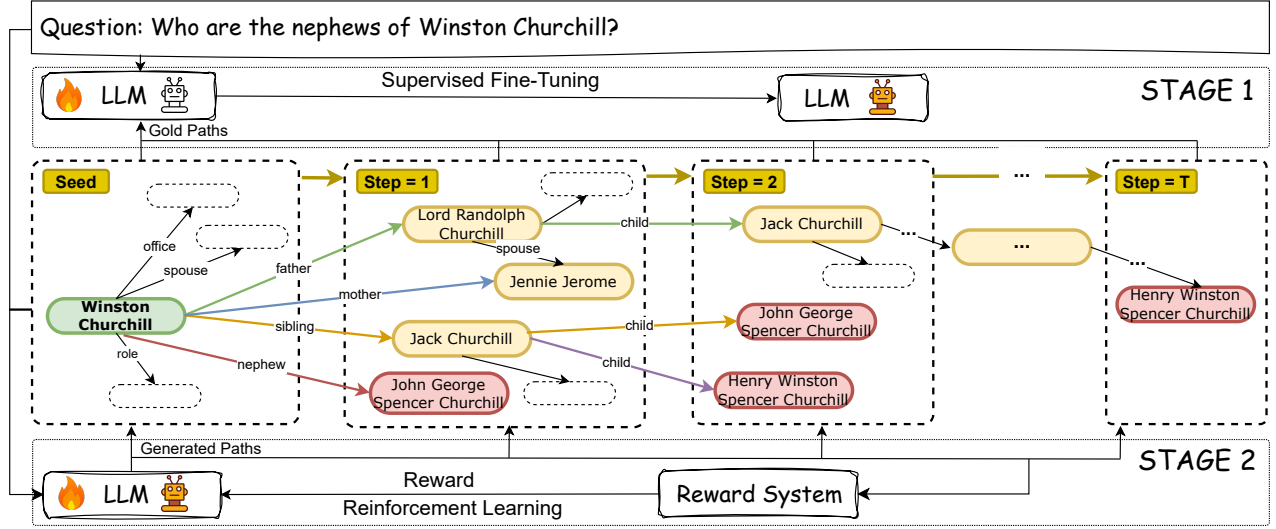


Figure 2: The framework of RoE. The model begins from the seed entity (green), incrementally expands to explored entities (yellow), while discovering answer entities (red). Stage 1 uses SFT to learn gold reasoning paths, and the trained model is then used as the initial model for Stage 2, where RL refines the exploration policy based on reward feedback.

representing the boundary entities that can be expanded at the next step. At step d , the model observes a **state**

$$s_d = (q, \mathcal{P}_d, \mathcal{N}_{\mathcal{F}_d}),$$

where $\mathcal{N}_{\mathcal{F}_d}$ denotes neighbor nodes and edges of all frontier nodes. The model then predicts an action a that consists of two lists:

- **Answers:** $\mathcal{A}_d = [ans_1^d, ans_2^d, \dots, ans_m^d]$, representing entities predicted as current answers at step d .
- **New Exploration Paths:** $\mathcal{P}_{d+1} = \{p_i^d \oplus (v_k, r_{k+1}, v_{k+1}) \mid p_i^d = [v_0, r_1, v_1, \dots, r_k, v_k] \in \mathcal{P}_d, (v_k, r_{k+1}, v_{k+1}) \in \mathcal{E}\}$, where $\oplus$ denotes the extension of an existing path with a newly explored triplet.

The exploration continues until the model predicts that no new paths should be expanded (i.e., $\mathcal{P}_{d+1} = \emptyset$) or a predefined maximum number of steps $D_{\max}$ is reached. The final outputs include all predicted answers $\mathcal{A} = \bigcup_d \mathcal{A}_d$, which enables the model to find multiple answers through reasoning paths of different lengths.

Through this design, retrieval and generation become mutually informed and jointly optimized, enabling LLMs to construct reasoning paths that are both complete and contextually relevant. However, empowering LLMs to perform such structured graph exploration introduces several challenges. First, LLMs are pretrained primarily on unstructured text and therefore lack the inherent ability to navigate and reason over graphs [9]. Second, naively training an LLM on exploration trajectories may cause it to memorize specific graph patterns rather than learning transferable exploration strategies. To address these challenges, we introduce a two-stage training strategy that equips the model with both graph exploration ability and strong generalization across different graphs.

4.2 Stage I: Supervised Fine-Tuning for Step-wise Exploration

In the first stage, we aim to equip the LLM with the basic ability to perform step-wise graph exploration. Specifically, we require the **gold actions** for each step, which include both the step-wise

answers and the new exploration paths as defined in Section 4.1. Previous works [46, 71] typically use the *shortest path* from the query entities to the answers to supervise their models. However, we argue that relying solely on the shortest path is overly restrictive. In real knowledge graphs, there may exist multiple valid reasoning paths connecting the same query entity to the correct answer; constraining training to only the shortest one discourages exploration and limits generalization. Moreover, the model does not have a global view of the entire graph at each step. When several neighboring entities share the same relation with the current frontier entity, selecting only a single target node can be ambiguous, especially when entity names are represented as IDs.

To address these issues, we construct the gold exploration paths by including all reasoning paths whose lengths are below a threshold, ensuring that the model learns to explore multiple plausible reasoning routes. Additionally, when one node is selected for expansion, other nodes connected to the frontier entity through the same relation are also treated as valid expansion candidates. For the answers at each step, we use the newly explored entities that correspond to the ground-truth answers as the gold answers. The detailed procedure is summarized in Algorithm 1 in Appendix A.

In the supervised fine-tuning (SFT) stage, each training instance is represented as a pair (s_d, a_d^*) in $\mathcal{D}_{\text{SFT}}$, where s_d denotes the state at step d (including the question, current reasoning paths, and local neighborhood), and a_d^* is the corresponding gold action constructed in Algorithm 1. At each step, the model takes s_d as input and predicts action $a_d = (\mathcal{A}_d, \mathcal{P}_d)$, which includes the step-wise answers and the new exploration paths. The training objective is to maximize the likelihood of the gold action a_d^* given its state, leading to loss:

$$\mathcal{L}_{\text{SFT}} = - \sum_d \log P(a_d^* \mid s_d; \theta),$$

where θ denotes the model parameters. This loss encourages the model to imitate the expert exploration behavior, thereby acquiring

the fundamental ability for structured graph exploration before reinforcement-learning refinement in Stage II.

4.3 Stage II: Reinforcement Learning for Exploration Optimization

Although the SFT stage enables the model to imitate gold trajectories, it may cause the LLM to memorize fixed exploration patterns within a single dataset, limiting its ability to generalize to unseen graphs or question types. To address this, we introduce a reinforcement learning (RL) stage that allows the model to interact with the knowledge graph and refine its exploration strategy through feedback rather than imitation. During this stage, we not only encourage the LLM to generate correct step-wise answers and exploration paths, but also aim to promote broader and more diverse exploration behaviors, enabling the model to discover additional valid reasoning trajectories and answers beyond those seen in the gold data. To enable such flexible and interpretable optimization, we adopt a rule-based reinforcement learning framework, which provides explicit feedback signals without relying on a learned value model. This choice is inspired by recent advances in reasoning LLMs, such as DeepSeek-R1 [15], which have shown that rule-based rewards can effectively guide reasoning and exploration, while simplifying training compared to the original RLHF framework [40].

In RoE, we expect the model to generate outputs in the correct format, align its predictions with the gold answers and exploration paths, and simultaneously discover new valid answers and paths while penalizing hallucinated ones. Therefore, we define the following five rewards:

Format Reward. Different from previous methods that rely on a “thinking” process, we directly prompt the LLM to generate both the step-wise answers and exploration paths in a structured JSON format to improve efficiency. Specifically, at each step the model is required to output:

```
{"answers": [], "exploration_paths": []}.
```

A non-zero reward $\mathcal{R}_{\text{format}}$ is given only when the model successfully follows the required format.

Answer Reward. We measure how many of the gold answers $\mathcal{A}_d^*$ in the current state s_d are successfully predicted by the model. Specifically, we compute the recall between the predicted and gold answer sets at each step d :

$$\mathcal{R}_{\text{ans}} = \frac{|\mathcal{A}_d^p \cap \mathcal{A}_d^*|}{|\mathcal{A}_d^*|},$$

where $\mathcal{A}_d^p$ denotes the predicted answers. A higher reward is assigned when more ground-truth answers are correctly identified.

Answer Discovery Reward. To promote broader and more effective exploration, we further encourage the model to discover new valid answers that were not present in the previous steps. Let $\mathcal{A}$ denote the set of all gold answers for the question. We reward the model for predicting new correct answers $\mathcal{A}_d^p \cap (\mathcal{A} \setminus \mathcal{A}_d^*)$ and penalize it for generating invalid ones $\mathcal{A}_d^p \setminus \mathcal{A}$:

$$\mathcal{R}_{\text{ans-dis}} = |\mathcal{A}_d^p \cap (\mathcal{A} \setminus \mathcal{A}_d^*)| - \beta \cdot |\mathcal{A}_d^p \setminus \mathcal{A}|,$$

where β is a penalty weight controlling the impact of hallucinated predictions. This reward encourages the model to find novel correct

answers, by leveraging its internal knowledge and the reasoning paths explored so far, while maintaining answer precision.

Exploration Reward. Similar to the answer reward, this reward evaluates how well the model’s predicted reasoning paths match the gold paths in the current state. Let $\mathcal{P}_d^p$ and $\mathcal{P}_d^*$ denote the predicted and gold reasoning paths at step d , respectively. We compute the reward as the recall between the predicted and gold path sets:

$$\mathcal{R}_{\text{explore}} = \frac{|\mathcal{P}_d^p \cap \mathcal{P}_d^*|}{|\mathcal{P}_d^*|}.$$

A higher reward indicates that the model successfully explores the correct reasoning paths in the current step, guiding it to perform faithful and consistent graph traversal.

Exploration Discovery Reward. To further promote effective and diverse reasoning, we encourage the model to explore new valid paths while penalizing invalid ones. Let $\mathcal{P}_d^p$ denote the set of predicted paths at step d , and $\mathcal{P}_d^*$ denote the set of gold paths. We define *new paths* as those that do not appear in $\mathcal{P}_d^*$ but exist on the knowledge graph $\mathcal{G}$, and *invalid paths* as those whose triplets are not present in $\mathcal{G}$. The reward is computed as:

$$\mathcal{R}_{\text{exp-dis}} = |\mathcal{P}_d^p \setminus \mathcal{P}_d^*|_{\text{valid}} - \beta \cdot |\mathcal{P}_d^p|_{\text{invalid}},$$

where $|\mathcal{P}_d^p \setminus \mathcal{P}_d^*|_{\text{valid}}$ counts the number of new but valid paths on the graph and $|\mathcal{P}_d^p|_{\text{invalid}}$ counts the number of paths containing nonexistent triplets. This reward encourages the model to explore novel and meaningful reasoning directions while discouraging invalid graph traversals.

Final Reward. The overall reward for each step is defined as the sum of all the individual rewards introduced above:

$$\mathcal{R}_{\text{total}} = \mathcal{R}_{\text{format}} + \mathcal{R}_{\text{ans}} + \mathcal{R}_{\text{ans-dis}} + \mathcal{R}_{\text{explore}} + \mathcal{R}_{\text{exp-dis}}.$$

This combined reward jointly encourages the model to produce well-structured outputs, generate correct answers, and discover diverse and valid reasoning paths during reinforcement learning. We optimize RoE using Group Relative Policy Optimization (GRPO) [15], a variant of Proximal Policy Optimization (PPO) [44].

In summary, RoE is first supervised through step-wise fine-tuning (Stage I) to learn the fundamental exploration behavior from gold trajectories, and is then refined via reinforcement learning (Stage II) to improve its reasoning and generalization ability. Through this two-stage training framework, RoE learns to jointly reason and explore over graphs, constructing accurate and diverse reasoning paths while producing faithful multi-hop answers.

5 Experiments

In this section, we conduct comprehensive experiments to validate the effectiveness of the proposed RoE. Specifically, we aim to answer the following research questions: **(1) RQ1:** How does RoE perform on benchmark datasets compared with baseline methods? **(2) RQ2:** Can RoE generalize to unseen datasets better than existing GraphRAG approaches? **(3) RQ3:** How do different design choices and factors influence the performance of RoE?

5.1 Experimental Settings

Datasets. To evaluate the effectiveness of RoE, we use two widely adopted datasets: **WebQSP** [67] and **CWQ** [49], both constructed

Table 2: Performance comparison on two KGQA benchmarks. The best and second-best are in bold and underline. * indicates the model uses Llama3.1-8B-Instruct as backbone.

Category	Method	WebQSP		CWQ		Avg. Rank
		Hits@1	F1	Hits@1	F1	
LLMs-only	Flan-T5-xl	31.00	-	14.70	-	12.00
	Alpaca-7B	51.80	-	27.40	-	11.00
	Llama-2-7B-Chat	64.40	-	34.60	-	8.50
	Llama-3.1-8B-Instruct*	58.96	31.66	31.49	20.66	10.25
	GPT-4.1	64.99	44.04	44.09	36.56	8.25
GNNs-only	GraftNet	-	62.40	-	32.70	8.00
	SR+NSM	-	64.10	-	47.10	5.50
	UniKGQA	-	70.20	-	48.00	3.50
LLMs+GNNs	G-Retriever*	74.07	54.51	51.51	45.18	6.75
	GNNRAG*	85.25	<u>72.01</u>	63.61	54.92	2.00
LLMs+KGs	ToG + Llama2-70B-Chat	63.70	-	53.60	-	7.50
	RoG*	78.62	64.39	53.83	45.97	5.00
	KD-CoT*	68.60	52.50	55.70	-	6.33
	SubGraphRAG*	<u>86.61</u>	70.57	56.98	47.16	3.00
	RoE*	89.13	74.76	66.49	<u>53.21</u>	1.25

upon the Freebase knowledge graph [4]. To further assess the generalization ability, we additionally use **MetaQA** [73], which is built on the WikiMovies KG [38], and **PathQuestion** [75], which uses a Freebase subset with modified relation names.

Evaluation Metrics. We follow previous works [16, 20, 31] and adopt two evaluation metrics: **Hit** and **F1**. Specifically, the Hit measures the proportion of questions for which at least one correct predicted answer, while F1 provides a balanced evaluation of both precision and recall.

Baselines. We compare RoE with a diverse set of baselines, which can be grouped into four categories: (1) **LLM-Only**: methods that rely solely on the internal knowledge of LLMs to answer questions without using external knowledge graphs. We include Flan-T5-XL [8], Alpaca-7B [50], Llama-2-7B-Chat [52], Llama-3.1-8B-Instruct [52], and GPT-4.1 [1]. (2) **GNN-Only**: models that use GNNs for both retrieval and reasoning. We include GraftNet [47], SR+NSM [71], and UniKGQA [24]. (3) **LLMs + GNNs**: hybrid approaches that combine LLMs and GNNs for retrieval and generation. For instance, G-Retriever [21] leverages GNNs to enhance LLM reasoning, while GNN-RAG [37] employs GNNs for retrieval and LLMs for generation. (4) **LLMs + KG**: methods that employ LLMs for both retrieval and generation over knowledge graphs. We compare with ToG [48], RoG [35], KD-CoT [54], and SubGraphRAG [31].

RoE settings. For RoE, we use the Llama-3.1-8B-Instruct model [52] as the backbone. We divide the training data of both WebQSP and CWQ into two parts: 60% is used for the SFT (stage I) and the remaining 40% for the RL (stage II). To reduce memory consumption during training, we apply the LoRA [22] in both stages.

To ensure a fair comparison, we train each retriever and generator on a single dataset, rather than jointly training across multiple datasets as done in RoG and GNN-RAG. Moreover, to eliminate the influence of backbone model, we use the same **Llama-3.1-8B-Instruct** model [52] for most LLM-based baselines, including RoG, KD-CoT, SubGraphRAG, G-Retriever, and GNN-RAG. We also evaluate with different backbone models in Appendix D.

5.2 Overall Performance Comparison

The overall performance on the WebQSP and CWQ datasets is shown in Table 2. We make the following observations:

- The proposed RoE achieves the best performance across most metrics on both datasets. Specifically, RoE attains relative improvements of 2.9% and 3.8% on Hit and F1, respectively, compared to the second-best models on the WebQSP dataset. On the CWQ dataset, RoE also achieves a 3.1% relative improvement in Hit, demonstrating its strong ability to reason on graphs.
- The LLM-only models, which lack access to graph information, generally perform worse than methods that incorporate graphs. However, stronger models, such as GPT-4.1, achieve better results, suggesting that large LLMs already possess internal knowledge relevant to the benchmarks. This observation justifies our choice that using the same LLM backbone for baselines to ensure a fair comparison.
- The GNN-only models generally perform worse than methods that incorporate LLMs and graph information, indicating that the internal knowledge and reasoning capability of LLMs are essential for solving multi-hop KGQA tasks.
- Compared with LLMs+GNNs methods, RoE achieves better performance in most cases, indicating additional GNN modules may not be necessary, suggesting LLMs themselves can acquire effective graph reasoning capabilities through fine-tuning.
- For the LLMs+KGs methods, such as ToG, KD-CoT, and SubGraphRAG, which prompt pretrained LLMs to iteratively retrieve and generate sub-questions or directly answer based on a retrieved subgraph, the performance is still lower than that of the proposed RoE. This suggests that pretrained LLMs are not well-suited for graph reasoning without additional fine-tuning.

5.3 Generalization Performance Comparison

In this section, we evaluate the generalization capability of RoE. Unlike previous works [35, 37], which fine-tune their models on the target graphs, we directly apply the pretrained model from one dataset to unseen datasets without any additional training. Specifically, we

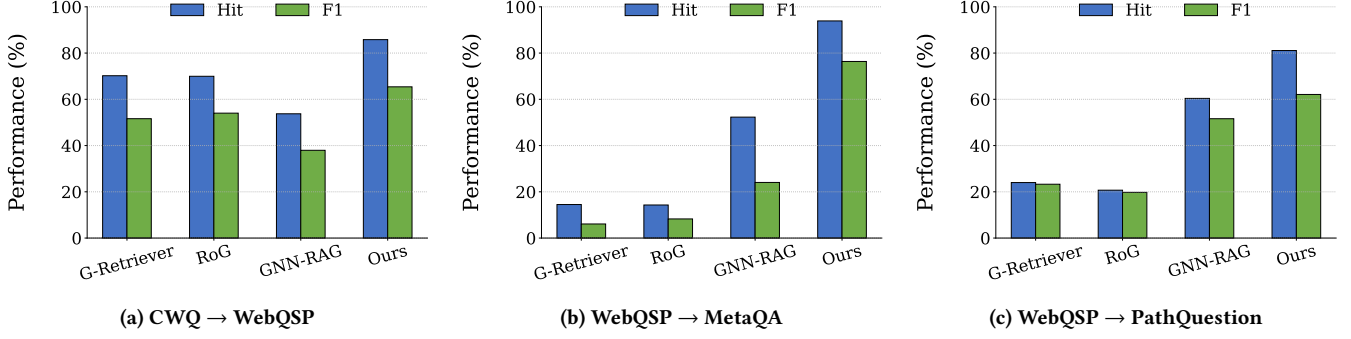


Figure 3: Generalization performance (Hit/F1) of different methods across dataset transfers.

select G-Retriever, RoG, and GNN-RAG as baselines, and evaluate the transfer performance under three settings: **CWQ → WebQSP**, **WebQSP → PathQuestion**, and **WebQSP → MetaQA**.

The generalization results are presented in Figure 3. As shown, the proposed RoE significantly outperforms all baselines across different transfer settings. For example, under the WebQSP → MetaQA transfer, RoE achieves a Hit of **93.90**, while the second-best method only reaches **52.30**, demonstrating the strong generalization capability of RoE across distributional shifts in both questions and knowledge graphs. Specifically, although G-Retriever employs a heuristic-based retriever without explicit retriever training, it still requires training a GNN model to encode the graph structure. As a result, G-Retriever performs well on the CWQ → WebQSP transfer setting, where both datasets share the same underlying knowledge graph, but performs poorly in other settings with different graph structures. Similarly, the RoG model relies on predicting reasoning relation paths, which are difficult to transfer across graphs with different relation vocabularies. GNN-RAG relies on trained GNNs for retrieval, which also makes it difficult to generalize to different question distributions and knowledge graphs. As a result, it tends to retrieve overly large subgraphs under transfer settings, as discussed in Section 3, thereby introducing substantial noise that negatively affects the generator’s reasoning performance.

5.4 Ablation Studies

In this subsection, we conduct ablation studies to verify the effectiveness of the key components in RoE. Specifically, we analyze the impact of the reinforcement learning (RL) stage and the proposed discovery rewards for both answers and exploration paths. We conduct experiments on two variants of RoE: (1) removing the reinforcement learning (RL) stage and using only supervised fine-tuning (SFT), denoted as **RoE w/o RL**; and (2) removing the discovery rewards for both Answer Discovery Reward and Exploration Discovery Reward during the RL stage, denoted as **RoE w/o Discovery**. We train these variants on WebQSP dataset and test their performance on WebQSP, MetaQA and PathQuestion dataset.

The results are presented in Figure 4, from which we make the following observations: **(1)** The RL stage is essential for RoE, especially under transfer settings. RoE w/o RL shows significantly lower performance on the MetaQA and PathQuestion datasets, demonstrating that models trained only with SFT tend to memorize dataset-specific patterns and struggle to generalize to unseen

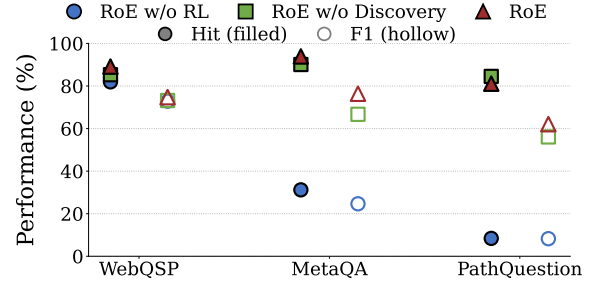


Figure 4: Performance of RoE and its variants pretrained on the WebQSP dataset. Marker shapes denote different model variants, while filled and hollow markers represent Hit and F1 metrics, respectively.

graphs without reinforcement learning. **(2)** The discovery rewards, which guide the model to find more valid answers and paths while penalizing hallucinations, are also crucial for performance. Without these rewards, the model tends to generate more hallucinated answers, resulting in a notable drop in F1 under transfer settings.

6 Conclusion

In this paper, we revisited the limitations of existing two-stage GraphRAG frameworks, where the retriever and generator are optimized independently and lack interaction during reasoning. We proposed **Reasoning by Exploration (RoE)**, a unified framework that integrates retrieval and generation into a step-wise exploration process. By allowing the model to interact with the graph dynamically and incorporate feedback from intermediate steps, RoE learns to reason and explore rather than rely on static retrieval patterns. Extensive experiments on multiple KGQA benchmarks demonstrate that RoE consistently outperforms strong baselines and generalizes effectively across different question distributions and graphs.

Acknowledgment

This work is supported by the National Science Foundation (NSF) under grant numbers CNS2321416, IIS2212032, IIS2212144, IIS2504089, DUE2234015, CNS2246050, DRL2405483, IIS 2524379, and NAIRR 250188, the Michigan Department of Agriculture and Rural Development, US Dept of Commerce, Gates Foundation, Amazon Faculty Award, Meta, NVIDIA, Microsoft and SNAP.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Tareq Al-Moslimi, Marc Gallofré Ocaña, Andreas L Opdahl, and Csaba Veres. 2020. Named entity extraction for knowledge graphs: A literature overview. *IEEE access* 8 (2020), 32862–32881.
- [3] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 17682–17690.
- [4] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. 2008. Freebase: a collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 1247–1250.
- [5] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.
- [6] Nurendra Choudhary and Chandan K Reddy. 2023. Complex logical reasoning over knowledge graphs using large language models. *arXiv preprint arXiv:2305.01157* (2023).
- [7] Yucheng Chu, Peng He, Hang Li, Haoyu Han, Kaiqi Yang, Yu Xue, Tingting Li, Joseph Krajcik, and Jiliang Tang. 2025. Enhancing LLM-Based Short Answer Grading with Retrieval-Augmented Generation. *arXiv preprint arXiv:2504.05276* (2025).
- [8] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.
- [9] Xinnan Dai, Haohao Qu, Yifen Shen, Bohang Zhang, Qihao Wen, Wenqi Fan, Dongsheng Li, Jiliang Tang, and Caihua Shan. 2024. How do large language models understand graph patterns? a benchmark for graph pattern comprehension. *arXiv preprint arXiv:2410.05298* (2024).
- [10] Xiang Deng, Yu Su, Alyssa Lees, You Wu, Cong Yu, and Huan Sun. 2021. ReasonBERT: Pre-trained to reason with distant supervision. *arXiv preprint arXiv:2109.04912* (2021).
- [11] Ritam Dutt, Kasturi Bhattarajee, Rashmi Gangadharai, Dan Roth, and Carolyn Rose. 2022. PerKGQA: Question answering over personalized knowledge graphs. In *Findings of the Association for Computational Linguistics: NAACL 2022*. 253–268.
- [12] Jinyuan Fang, Zaiqiao Meng, and Craig Macdonald. 2024. Reano: Optimising retrieval-augmented reader models through knowledge graph generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2094–2112.
- [13] Yanlin Feng, Xinyue Chen, Bill Yuchen Lin, Peifeng Wang, Jun Yan, and Xiang Ren. 2020. Scalable multi-hop relational reasoning for knowledge-aware question answering. *arXiv preprint arXiv:2005.00646* (2020).
- [14] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [15] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [16] Kai Guo, Harry Shomer, Shenglai Zeng, Haoyu Han, Yu Wang, and Jiliang Tang. 2025. Empowering graphrag with knowledge filtering and integration. *arXiv preprint arXiv:2503.13804* (2025).
- [17] Haoyu Han, Li Ma, Harry Shomer, Yu Wang, Yongjia Lei, Kai Guo, Zhigang Hua, Bo Long, Hui Liu, Charu C Aggarwal, et al. 2025. Rag vs. graphrag: A systematic evaluation and key insights. *arXiv preprint arXiv:2502.11371* (2025).
- [18] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309* (2024).
- [19] Haoyu Han, Yaochen Xie, Hui Liu, Xianfeng Tang, Sreyashi Nag, William Headen, Yang Li, Chen Luo, Shuiwang Ji, Qi He, et al. 2025. Reasoning with graphs: Structuring implicit knowledge to enhance llms reasoning. *arXiv preprint arXiv:2501.07845* (2025).
- [20] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. 2021. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM international conference on web search and data mining*. 553–561.
- [21] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems* 37 (2024), 132876–132907.
- [22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [23] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. 2024. Kg-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. *arXiv preprint arXiv:2402.11163* (2024).
- [24] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. *arXiv preprint arXiv:2212.00959* (2022).
- [25] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 7969–7992.
- [26] Enkelejd Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and individual differences* 103 (2023), 102274.
- [27] Seyed Mehran Kazemi and David Poole. 2018. Simple embedding for link prediction in knowledge graphs. *Advances in neural information processing systems* 31 (2018).
- [28] Yongjia Lei, Haoyu Han, Ryan A. Rossi, Franck Dernoncourt, Nedim Lipka, Mahantesh M Halappanavar, Jiliang Tang, and Yu Wang. 2025. Mixture of Structural-and-Textual Retrieval over Text-rich Graph Knowledge Bases. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria.
- [29] Yongjia Lei, Haoyu Han, Ryan A Rossi, Franck Dernoncourt, Nedim Lipka, Mahantesh M Halappanavar, Jiliang Tang, and Yu Wang. 2025. Mixture of Structural-and-Textual Retrieval over Text-rich Graph Knowledge Bases. *arXiv preprint arXiv:2502.20317* (2025).
- [30] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [31] Mufei Li, Siqi Miao, and Pan Li. 2024. Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation. *arXiv preprint arXiv:2410.20724* (2024).
- [32] Guangyi Liu, Yongqi Zhang, Yong Li, and Quanming Yao. 2024. Dual reasoning: A gnn-llm collaborative framework for knowledge graph question answering. *arXiv preprint arXiv:2406.01145* (2024).
- [33] Elita Lobo, Chirag Agarwal, and Himabindu Lakkaraju. 2024. On the impact of fine-tuning on chain-of-thought reasoning. *arXiv preprint arXiv:2411.15382* (2024).
- [34] Haoran Luo, Guanting Chen, Qika Lin, Yikai Guo, Fangzhi Xu, Zemin Kuang, Meina Song, Xiaobao Wu, Yifan Zhu, Luu Anh Tuan, et al. 2025. Graph-r1: Towards agentic graphrag framework via end-to-end reinforcement learning. *arXiv preprint arXiv:2507.21892* (2025).
- [35] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2023. Reasoning on graphs: Faithful and interpretable large language model reasoning. *arXiv preprint arXiv:2310.01061* (2023).
- [36] Yao Ma and Jiliang Tang. 2021. *Deep learning on graphs*. Cambridge University Press.
- [37] Costas Mavromatis and George Karypis. 2024. Gnn-rag: Graph neural retrieval for large language model reasoning. *arXiv preprint arXiv:2405.20139* (2024).
- [38] Alexander Miller, Adam Fisch, Jesse Dodge, Amir-Hossein Karimi, Antoine Bordes, and Jason Weston. 2016. Key-value memory networks for directly reading documents. *arXiv preprint arXiv:1606.03126* (2016).
- [39] Marianna Nezhurina, Lucia Ciolina-Kun, Mehdi Cherti, and Jenia Jitsev. 2024. Alice in wonderland: Simple tasks showing complete reasoning breakdown in state-of-the-art large language models. *arXiv preprint arXiv:2406.02061* (2024).
- [40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [41] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921* (2024).
- [42] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.
- [43] Diego Sanmartin. 2024. Kg-rag: Bridging the gap between knowledge and creativity. *arXiv preprint arXiv:2405.12035* (2024).
- [44] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*

- (2017).
- [45] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, et al. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419* (2025).
 - [46] Haitian Sun, Tania Bedrax-Weiss, and William W Cohen. 2019. Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. *arXiv preprint arXiv:1904.09537* (2019).
 - [47] Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William W Cohen. 2018. Open domain question answering using early fusion of knowledge bases and text. *arXiv preprint arXiv:1809.00782* (2018).
 - [48] Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. *arXiv preprint arXiv:2307.07697* (2023).
 - [49] Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. *arXiv preprint arXiv:1803.06643* (2018).
 - [50] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Stanford alpaca: An instruction-following llama model.
 - [51] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language models in medicine. *Nature medicine* 29, 8 (2023), 1930–1940.
 - [52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
 - [53] Chaojie Wang, Yishi Xu, Zhong Peng, Chenxi Zhang, Bo Chen, Xinrun Wang, Lei Feng, and Bo An. 2023. keqing: knowledge-based question answering is a nature chain-of-thought mentor of LLM. *arXiv preprint arXiv:2401.00426* (2023).
 - [54] Keheng Wang, Feiyu Duan, Sirui Wang, Peiguang Li, Yunsen Xian, Chuantao Yin, Wenge Rong, and Zhang Xiong. 2023. Knowledge-driven cot: Exploring faithful reasoning in llms for knowledge-intensive question answering. *arXiv preprint arXiv:2308.13259* (2023).
 - [55] Xiaoyang Wang, Yao Ma, Yiqi Wang, Wei Jin, Xin Wang, Jiliang Tang, Caiyan Jia, and Jian Yu. 2020. Traffic flow prediction via spatial temporal graph neural network. In *Proceedings of the web conference 2020*. 1082–1092.
 - [56] Yu Wang, Nedim Lipka, Ryan A Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. 2024. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 19206–19214.
 - [57] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
 - [58] Yilin Wen, Zifeng Wang, and Jimeng Sun. 2023. Mindmap: Knowledge graph prompting sparks graph of thoughts in large language models. *arXiv preprint arXiv:2308.09729* (2023).
 - [59] Nirmalie Wiratunga, Ramitha Abeyratne, Lasal Jayawardena, Kyle Martin, Stewart Massie, Ikechukwu Nkisi-Orji, Ruwan Weerasinghe, Anne Lirer, and Bruno Fleisch. 2024. CBR-RAG: case-based reasoning for retrieval augmented generation in LLMs for legal question answering. In *International Conference on Case-Based Reasoning*. Springer, 445–460.
 - [60] Yike Wu, Nan Hu, Sheng Bi, Guilin Qi, Jie Ren, Anhuan Xie, and Wei Song. 2023. Retrieve-rewrite-answer: A kg-to-text enhanced llms framework for knowledge graph question answering. *arXiv preprint arXiv:2309.11206* (2023).
 - [61] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
 - [62] Fengli Xu, Qianyu Hao, Zefang Zong, Jingwei Wang, Yunke Zhang, Jingyi Wang, Xiaochong Lan, Jiahui Gong, Tianjian Ouyang, Fanjin Meng, et al. 2025. Towards large reasoning models: A survey of reinforced reasoning with large language models. *arXiv preprint arXiv:2501.09686* (2025).
 - [63] Ran Xu, Wenqi Shi, Yue Yu, Yuchen Zhuang, Bowen Jin, May D Wang, Joyce C Ho, and Carl Yang. 2024. Ram-ehr: Retrieval augmentation meets clinical predictions on electronic health records. *arXiv preprint arXiv:2403.00815* (2024).
 - [64] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
 - [65] Michihiro Yasunaga, Antoine Bosselut, Hongyu Ren, Xikun Zhang, Christopher D Manning, Percy S Liang, and Jure Leskovec. 2022. Deep bidirectional language-knowledge graph pretraining. *Advances in Neural Information Processing Systems* 35 (2022), 37309–37323.
 - [66] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with language models and knowledge graphs for question answering. *arXiv preprint arXiv:2104.06378* (2021).
 - [67] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 201–206.
 - [68] Chuanyue Yu, Kuo Zhao, Yuhua Li, Heng Chang, Mingjian Feng, Xiangzhe Jiang, Yufei Sun, Jia Li, Yuzhi Zhang, Jianxin Li, et al. 2025. GraphRAG-R1: Graph Retrieval-Augmented Generation with Process-Constrained Reinforcement Learning. *arXiv preprint arXiv:2507.23581* (2025).
 - [69] Boyu Zhang, Hongyang Yang, Tianyu Zhou, Muhammad Ali Babar, and Xiao-Yang Liu. 2023. Enhancing financial sentiment analysis via retrieval augmented large language models. In *Proceedings of the fourth ACM international conference on AI in finance*. 349–356.
 - [70] Dan Zhang, Sining Zhou, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. 2024. Rest-mcts*: Llm self-training via process reward guided tree search. *Advances in Neural Information Processing Systems* 37 (2024), 64735–64772.
 - [71] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. 2022. Subgraph retrieval enhanced model for multi-hop knowledge base question answering. *arXiv preprint arXiv:2202.13296* (2022).
 - [72] Xikun Zhang, Antoine Bosselut, Michihiro Yasunaga, Hongyu Ren, Percy Liang, Christopher D Manning, and Jure Leskovec. 2022. Greaselm: Graph reasoning enhanced language models for question answering. *arXiv preprint arXiv:2201.08860* (2022).
 - [73] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander J Smola, and Le Song. 2018. Variational Reasoning for Question Answering with Knowledge Graph. In *AAAI*.
 - [74] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. 2023. Large language models are not robust multiple choice selectors. *arXiv preprint arXiv:2309.03882* (2023).
 - [75] Mantong Zhou, Minlie Huang, and Xiaoyan Zhu. 2018. An interpretable reasoning network for multi-relation question answering. *arXiv preprint arXiv:1801.04726* (2018).
 - [76] Yuqi Zhu, Shuofei Qiao, Yixin Ou, Shumin Deng, Shiwei Lyu, Yue Shen, Lei Liang, Jinjie Gu, Huajun Chen, and Ningyu Zhang. 2024. Knowagent: Knowledge-augmented planning for llm-based agents. *arXiv preprint arXiv:2403.03101* (2024).

A Algorithm of SFT Dataset Construction

In this section, we detail the algorithm to construct the SFT datasets to train RoE. The algorithm is summarized in Algorithm 1, where lines 1–14 construct all gold-consistent reasoning paths up to length $L_{\max}$; line 20 derives the step-wise gold answers; and lines 21–22 generate the step-wise new exploration paths.

B Datasets

In the experiments, we adopt four different datasets, i.e. **WebQSP** [67] and **CWQ** [49], **MetaQA** [73] and **PathQuestion** [75]. For the WebQSP and CWQ dataset, we leverage the processed dataset in Luo et al. [35]. For the PathQuestion and MetaQA dataset, we randomly sample 1,000 samples of two hop questions. The statistics of these datasets are shown in Table 3.

Table 3: Statistics of datasets.

Datasets	#Train	#Test	Max #hop
WebQSP	2,826	1,628	2
CWQ	27,639	3,531	4
PathQuestion	0	1,000	2
MetaQA	0	1,000	2

C Experimental Settings

In this section, we present the detailed experimental settings for RoE. For RoE and most of baseline models involving LLMs, we use the **Llama-3.1-8B-Instruct** model [52] as the backbone. We first leverage Algorithm 1 to construct the gold trajectories for supervised fine-tuning. The training data of both WebQSP and CWQ are divided into two parts: 60% for the **Supervised Fine-Tuning (SFT)** stage and the remaining 40% for the **Reinforcement Learning (RL)** stage. To reduce memory consumption, we adopt **LoRA** [22] in both stages with a rank of 32.

During SFT, the loss is calculated only on the action prediction component, and the model is optimized using the AdamW optimizer with a learning rate of 1×10^{-4} . For the RL stage, we apply a smaller learning rate of 5×10^{-6} and use the Group Relative Policy Optimization (GRPO) algorithm to optimize the policy with rule-based rewards. For each sample, we generate 4 responses. Each training step uses a batch size of 4 with gradient accumulation to fit within GPU memory. Training is performed on 2×H200 GPUs.

During inference, we follow a depth-first exploration strategy. Because each node in the knowledge graph may have numerous neighbors that cannot fit into the LLM context window at once, we split the neighbors into multiple batches and iteratively feed them to the model. We use a depth-first search algorithm to explore the graph, and we set the maximum exploration depth to 5 hops to prevent overly long reasoning chains and context overflow. For reproducibility, all random seeds are fixed, and the same prompt templates for SFT and RL.

D Results with different backbone models

To evaluate robustness across model sizes, we additionally train RoE-3B and other baseline models with Llama-3.2-3B-Instruct backbone. All models are trained on WebQSP and evaluated on both WebQSP and MetaQA.

Algorithm 1 SFT Dataset Construction

Input: Question q ; KG $\mathcal{G} = \{\mathcal{V}, \mathcal{R}, \mathcal{E}\}$; seed entities $\mathcal{V}_0$; gold answers $\mathcal{A}^*$; max path length $L_{\max}$
Output: Dataset $\mathcal{D}_{\text{SFT}} = \{(s_d, a_d^*)\}_{d=0}^{L_{\max}-1}$ with $a_d^* = (\mathcal{A}_d^*, \mathcal{P}_d^*)$

- 1: **Phase I: Mine all gold-consistent paths (length $\leq L_{\max}$)**
- 2: $\Pi \leftarrow \emptyset$; $Q \leftarrow \{[v_0] \mid v_0 \in \mathcal{V}_0\}$
- 3: **while** Q not empty **do**
- 4: $p \leftarrow \text{pop from } Q$; $v \leftarrow \text{frontier}(p)$
- 5: **if** $|p| - 1 > L_{\max}$ **then continue**
- 6: **end if**
- 7: **if** $v \in \mathcal{A}^*$ **then** $\Pi \leftarrow \Pi \cup \{p\}$
- 8: **end if**
- 9: **for all** $(v, r, u) \in \mathcal{E}$ **do**
- 10: **if** $u \notin p$ **then** ▷ simple-path constraint (avoid cycles)
- 11: $Q \leftarrow Q \cup \{p \oplus (v, r, u)\}$
- 12: **end if**
- 13: **end for**
- 14: **end while**
- Phase II: Build step-wise actions from mined paths**
- 15: Define depth-indexed prefix pools for $d = 0, \dots, L_{\max}$:
 $\text{Pref}(d) = \{ \text{all prefixes of any } p \in \Pi \text{ with } |\text{prefix}| - 1 = d \}$
- 16: $\mathcal{D}_{\text{SFT}} \leftarrow \emptyset$
- 17: **for** $d = 0$ **to** $L_{\max} - 1$ **do**
- 18: $\mathcal{P}_d \leftarrow \text{Pref}(d)$; $\mathcal{F}_d \leftarrow \{\text{frontier}(p) \mid p \in \mathcal{P}_d\}$
- 19: **State:** $s_d \leftarrow (q, \mathcal{P}_d, \mathcal{N}_{\mathcal{F}_d})$
- 20: **Gold step-answers:**
 $\mathcal{A}_d^* = \{u \in \mathcal{A}^* \mid \exists (v, r, u) \in \mathcal{N}_{\mathcal{F}_d}\}$
- 21: **Gold one-hop extensions:**
 $\mathcal{E}_d^\Pi = \{(p, (v, r, u)) \in \text{Pref}(d+1) \mid p \in \mathcal{P}_d, \text{frontier}(p) = v\}$
- 22: **Same-relation sibling expansion:**
 $\mathcal{P}_d^* = \{p \oplus (v, r, u') \mid (p, (v, r, u)) \in \mathcal{E}_d^\Pi, (v, r, u') \in \mathcal{E}\}$
- 23: **Record step:** $a_d^* \leftarrow (\mathcal{A}_d^*, \mathcal{P}_d^*)$; $\mathcal{D}_{\text{SFT}} \leftarrow \mathcal{D}_{\text{SFT}} \cup \{(s_d, a_d^*)\}$
- 24: **end for**
- 25: **return** $\mathcal{D}_{\text{SFT}}$

Table 4: Performance comparison on the WebQSP and MetaQA datasets. All models use the Llama-3.2-3B-Instruct backbone and are trained on the WebQSP dataset.

	WebQSP $\rightarrow$ WebQSP		WebQSP $\rightarrow$ MetaQA	
Method	Hit	F1	Hit	F1
G-Retrieval-3B	67.32	47.25	12.7	5.58
RoG-3B	73.95	55.79	10.80	6.00
GNN-RAG-3B	81.51	67.16	41.70	13.94
RoE-3B	85.56	70.62	74.20	57.12

From the results in Table 4, RoE continues to outperform all baselines by a large margin, especially in the transfer setting on MetaQA. Notably, RoE-3B surpasses baselines even when they use Llama-8B, especially on the transfer setting. GRPO vs. GSPO yields similar performance, showing that RoE is not sensitive to the RL algorithm choice.